Comparison of Array-Based List and Linked List Implementations

Introduction

In this assignment, the List Abstract Data Type (ADT) was implemented using two different data structures: an **array-based list** and a **singly linked list**. The main objective was to understand how the same logical operations of a list can be implemented using different underlying data structures and how these choices affect performance, memory usage, and implementation complexity.

Both implementations support common list operations such as insertion, deletion, searching, reversing, and sorting. However, their internal working principles are very different, which leads to different advantages and disadvantages.

Array-Based List Implementation

The array-based list uses a **dynamic array** to store elements in contiguous memory locations. Initially, a fixed amount of memory is allocated. When the array becomes full, a larger array is created, existing elements are copied into it, and the old array is deallocated. This process is known as **dynamic resizing**.

Characteristics

- Elements are stored in **contiguous memory**
- Supports **direct access** using index
- Requires **shifting elements** during insertion and deletion
- Needs **resizing logic** to handle overflow and underflow

Time Complexity

- Access (get): **O(1)**
- Search: **O(n)**
- Insertion at end (amortized): **O(1)**
- Insertion at beginning or middle: **O(n)**
- Deletion: **O(n)** (due to shifting)

Advantages

- Fast access using index
- Better cache performance due to contiguous memory
- Less memory overhead (no extra pointer per element)

Disadvantages

- Insertion and deletion are costly
- Resizing is expensive due to copying
- Possible memory wastage when capacity is larger than size

Linked List Implementation

The linked list implementation uses a **singly linked list**, where each node contains two parts: data and a pointer to the next node. Nodes are stored in **non-contiguous memory locations** and are connected using pointers.

Characteristics

- Dynamic memory allocation for each node
- No need for resizing
- Sequential access only (no direct indexing)
- Extra memory needed for pointer storage

Time Complexity

- Access (get): **O(n)**
- Search: **O(n)**
- Insertion at beginning: **O(1)**
- Insertion at end: **O(n)** (without tail pointer)
- Deletion at beginning: **O(1)**
- Deletion at end or middle: **O(n)**

Advantages

- Efficient insertion and deletion (especially at head)
- No memory wastage due to resizing
- Flexible size (grows and shrinks dynamically)

Disadvantages

- Slower access due to sequential traversal
- Extra memory required for pointers
- Poor cache performance compared to arrays

Performance and Memory Comparison

Aspect	Array-Based List	Linked List
Memory Layout	Contiguous memory	Non-contiguous memory
Access by Index	O(1), very fast	O(n), sequential access
Insertion at Beginning	O(n)	O(1)
Insertion at End	O(1)	O(n)
Deletion	O(n)	O(1) at head, O(n) otherwise
Memory Overhead	Low	Higher (extra pointer per node)
Cache Performance	Better	Poorer
Resizing Requirement	Required	Not required

From a performance perspective, array-based lists are more suitable when **frequent access by index** is required, while linked lists are better when **frequent insertions and deletions** are needed.

Conclusion

Both array-based lists and linked lists have their own strengths and weaknesses. The array-based list provides faster access and better memory locality but suffers from costly insertions, deletions, and resizing overhead. On the other hand, the linked list offers efficient insertion and deletion operations and flexible memory usage but requires additional memory for pointers and slower access time.

Therefore, the choice between an array-based list and a linked list depends on the specific application requirements. If fast access and compact memory usage are more important, an array-based list is preferable. If dynamic size and frequent insertions or deletions are required, a linked list is a better choice.